Milestone-3 Design Doc

Demo Setup + Execution (All the details are provided in Readme inside the Milestone-3 Folder of main branch in GitHub)

To Replicate Demo Results:

Requirements

- At least 10 GB disk space & 16GB RAM recommended

Git Branch used for cloning the repo to Install and Build all the dependencies

pchintar-course-clean

Files to use for testing

Two Files available inside the Milestone-3 Folder of the main branch in GitHub:-
inMemoryCache_comparision.txt, countif_mode_testing.txt
'''

What the build does

1. Installs Java 17, Rust, Maven
2. Downloads Spark 3.5.8
3. Builds Comet from source

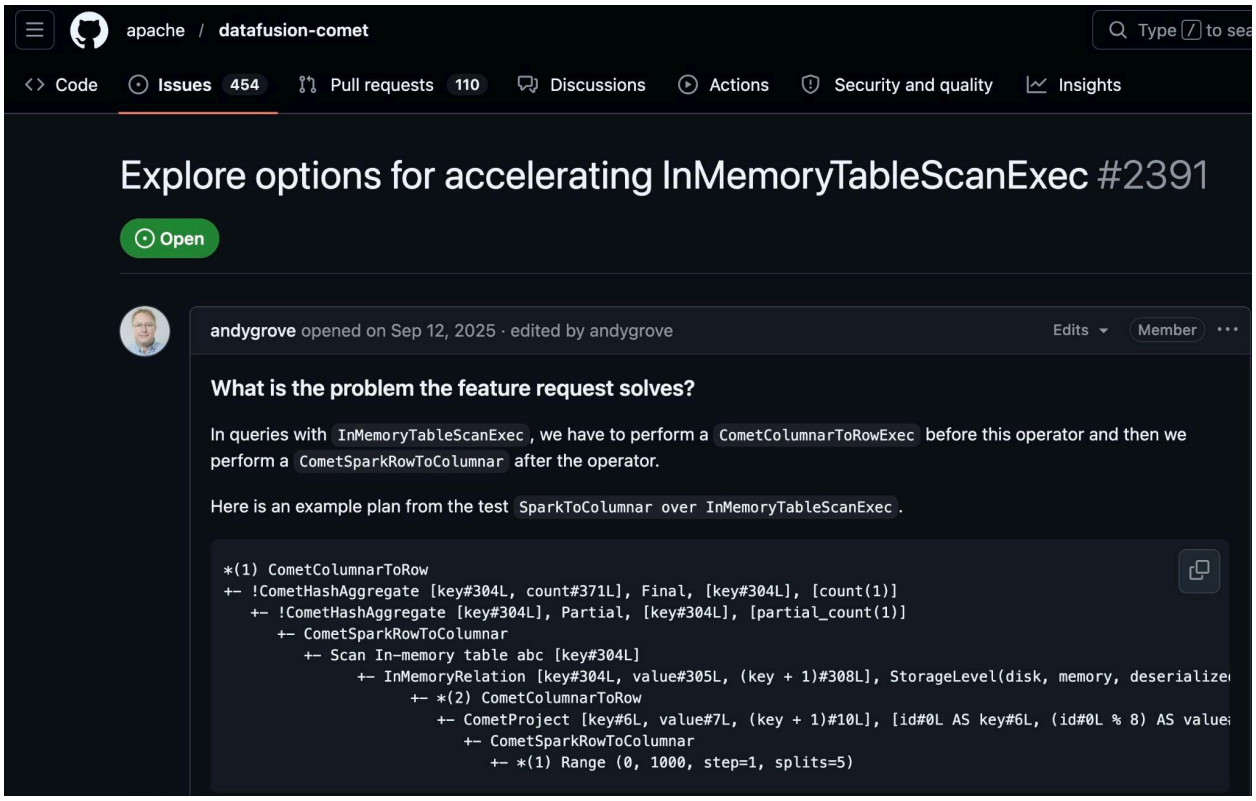
Our Aim for this Milestone

For the Milestone-3, we focused on non-benchmark related operator and sql function optimization by providing native comet support to certain operations that were previously unsupported and used to fall back to spark's JVM. We chose non-benchmark related issues for milestone-3 primarily due to the reasons below:

1) There are several senior Apache Datafusion engineers already working exactly on what we planned to work on w.r.t the benchmarks(TPC-H, TPC-DS) based operations and their commits are very rapid and directly solved several of the problems we wanted to solve ourselves very recently.
2) Legacy benchmarks such as TPC-H & TPC-DS are at least 15 years and so the queries provided in the benchmark are not always the most optimal way to get results especially with the recent advancements in new aggregate functions w.r.t sophistication & efficiency.

So, we focused on Github issues that were not directly related to the benchmark query optimization for this milestone. Below is the first one:

Milestone-3 Design Doc



Official GitHub issue link: [Explore options for accelerating InMemoryTableScanExec #2391]
Note: *Andy Grove* is the Original creator of Apache Datafusion as well as a leading senior contributor to Comet.

Problem

Currently, Comet doesn't provide native support for In-Memory cached data scanning or writing to cache. Also, Comet requires data to be in **Columnar Arrow format** while Spark's default cache serializer stores data in a **binary row-based format** within the JVM Heap. Therefore, when executing queries on cached data, Comet is forced to convert columnar data to rows before writing to cache (using Spark's row-based 'DefaultCachedBatchSerializer'), then convert rows back to columnar after reading from cache. This creates an extra & inefficient "conversion sandwich" of '***CometColumnarToRow***' and '***CometSparkRowToColumnar***' operators around '***InMemoryTableScanExec***', causing unnecessary data movement and CPU and memory overhead.

Milestone-3 Design Doc

Our Solution

Eliminating Spark↔Comet Conversion Overhead

The core mechanism is to **use a Comet-controlled, Arrow-based columnar cache representation directly during reads**, instead of routing cached data through Spark's default row-oriented in-memory scan path.

We created a new configuration flag called '***spark.comet.exec.inMemoryCache.enabled***' that when enabled eliminates the repeated row←→columnar data format conversions for In-Memory data and allows '***CometInMemoryTableScanExec***' to read the cached columnar data natively.

Approach

Our solution consists of two coordinated components:

1. Cache Write Path

Cached data is stored in a **Comet-specific format backed by serialized Arrow batches**, rather than Spark's default cached-batch representation.

- Input data (row or columnar) is converted into **Arrow columnar batches**
- These batches are serialized and stored as **CometCachedBatch**
- This ensures cached data is already in a **columnar, Comet-compatible form**

---

2. Cache Read Path

Cached data is read through a **Comet-specific scan operator**, bypassing Spark's default `InMemoryTableScanExec`.

- Cached payload is **decoded directly into columnar batches**
- Output is immediately usable by Comet operators
- No intermediate row-based processing occurs

During read, only the **columns required by the query** are selectively materialized from the cache data to reduce unnecessary data movement

---

Milestone-3 Design Doc

Results

SQL QUERY

```
// Andy Grove's test
spark.range(1000)
  .selectExpr("id as key", "id % 8 as value")
  .toDF("key", "value")
  .selectExpr("key", "value", "key+1")
  .createOrReplaceTempView("abc")

spark.catalog.cacheTable("abc")

val df = spark.sql("SELECT * FROM abc").groupBy("key").count()
df.explain("formatted")
df.collect()
```

Before(Default approach):

- Cache = Spark format
- Plan: Scan In-Memory table → CometSparkColumnarToColumnar → Comet Operations
- Several extra conversion steps from RowtoColumnar & Vice-versa → extra overhead

```
scala> df.explain("formatted")
== Physical Plan ==
* CometColumnarToRow (10)
+- CometHashAggregate (9)
   +- CometHashAggregate (8)
      +- CometSparkColumnarToColumnar (7)
         +- Scan In-memory table abc (1)
               +- InMemoryRelation (2)
                     +- * CometColumnarToRow (6)
                        +- CometProject (5)
                           +- CometSparkRowToColumnar (4)
                              +- * Range (3)
```

Milestone-3 Design Doc

After(When our new inMemory cache configuration flag is enabled):

- Cache = Comet format
- Plan: CometInMemoryScan → Comet Operations
- No conversion steps and highly optimal/efficient execution → Much reduced total steps

```
scala> df.explain("formatted")
== Physical Plan ==
* CometColumnarToRow (4)
+- CometHashAggregate (3)
   +- CometHashAggregate (2)
      +- CometInMemoryTableScan (1)
```

Next, for SQL aggregation functions `count_if` **and** `mode`, we implemented them as native aggregates in Comet so they execute without falling back to Spark as they were not supported by Comet. `count_if` maintains a **single running counter** that increments when the condition is true, making it simple and constant-memory. In contrast, `mode` tracks a **frequency table (value → count)** to determine the most frequent value, which requires maintaining structured state. We utilized the Hash table data structure due to its efficiency in lookup speed. To support this within Comet's execution model, the frequency table is stored as a **single compact state representation**, allowing it to be passed and combined correctly across execution stages. This ensures both functions run fully in Comet while preserving correctness and efficiency. We tested using diverse edge cases to check whether the results are indeed as per expectations with regards to accuracy and all those edge case results are accurate.

Future Work

Currently, due to system limitations of available RAM & Disk space, we're unable to benchmark on big data to get the measurable improvements in latency. We had to do quite a lot of debugging as well as re-building using both Cargo and Maven install because of our changes to the .rs files as well as .scala and .proto files. So, next for the final report, we'll be focusing and plan on using the freely available but limited Cloud credits to benchmark on the 100GB to even 1TB data to measure latency and memory performance.